handler will be invoked for all property authorization requirements, check the `Name` property of the requirement to determine which specific requirement is being evaluated. If the `Edit` requirement is being evaluated, check the current user's `NameIdentifier` claim for their `Id`, and compare it to the resource's `CreatorId` property. If there is a match, the current user created the resource, so you should mark the requirement as succeeded.

You have also decided that only users who are in the `Admin` role can delete a property. So if the current requirement is the `Delete` requirement, only mark it as successful under those circumstances. You don't actually need to access the resource to make this evaluation, but it makes sense to keep all the authorization logic for a resource in one place. The handler's code is as shown in the following listing.

Listing 10.26 The `PropertyAuthorizationHandler` class

```
using CityBreaks.AuthorizationRequirements;
using CityBreaks.Models;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Authorization.Infrastructure;
using System.Security.Claims;

namespace CityBreaks.AuthorizationHandlers
{
    public class PropertyAuthorizationHandler :
        AuthorizationHandler<OperationAuthorizationRequirement, Property>     ❶
    {
        protected override Task                                                ❷
        ➥ HandleRequirementAsync(AuthorizationHandlerContext context,         ❷
            OperationAuthorizationRequirement requirement, Property resource)❷
        {
            if(requirement.Name == PropertyOperations.Edit.Name)               ❸❹
            {                                                                  ❸
                if (resource.CreatorId ==                                      ❸
                ➥ context.User.FindFirst(c => c.Type ==                       ❸
                ➥ ClaimTypes.NameIdentifier).Value)                           ❸
                {                                                              ❸
```